

# Mycellius — Séance 9 (4h)

*Mycellius Mobile React Native (Expo)*  
*Onboarding/Login avec token SecureStore (token stocké en Secure Storage) +*  
*Pages (liste / détail / recherche)*

## Objectif

- On reproduit sur mobile ce que vous avez déjà fait sur le front web et on obtient une app mobile fonctionnelle.

## Objectif technique:

- appeler POST /api/v1/auth/login pour obtenir un JWT
- stocker ce token côté client (mais sur mobile, on évite localStorage → SecureStore)
- consommer l'API Pages (liste + détail + recherche) avec Authorization: Bearer <token>

## Pré-requis côté API (déjà dans vos séances 6 → 8)

- Endpoints pages “front-ready” (liste paginée + recherche titre)
- Auth JWT : POST /api/v1/auth/login retourne un token, et /api/v1/pages/\*\* exige un token
- Variante d'API possible pour la liste : soit Page<> (content), soit liste simple (array)

## Préconditions (10–15 min) — “page témoin” + API OK

But : éviter “ça ne marche pas sur mobile” alors que l'API n'est pas OK.

### 0.1 Démarrer MySQL (Docker)

Commande (racine du repo) :

```
docker compose -f infra/db/docker-compose.yml up -d
```

=> Attendu : le container MySQL est “Up”.

### 0.2 Démarrer l'API

```
mvn -f api/pom.xml spring-boot:run
```

=> Attendu : /api/health répond.

### 0.3 Créer (ou vérifier) une page témoin

POST <http://localhost:8080/api/v1/pages>

puis GET immédiat : <http://localhost:8080/api/v1/pages/{ID}>

### 0.4 Tester login (preuve JWT)

POST <http://localhost:8080/api/v1/auth/login>

avec un compte seed (ex: admin).

=> Attendu : 200 + token.

## Checkpoint fin préconditions :

=> API OK, login OK, une page existe en base.

## Bloc A (45 min) — Installer React Native “sans pièges” (Expo)

### A1 Installer Node.js (sur poste)

Node 18+ recommandé (comme pour votre front Vite).

Commandes de vérification :

```
node -v  
npm -v
```

### A2 Choisir la stratégie d'exécution mobile

**Option 1 (Ce qu'on va faire) : smartphone + appli “Expo Go”**

Avantage : pas d'Android Studio, pas d'émulateur, vous gagnez 30–45 min.

**Option 2 : émulateur Android (pour votre connaissance)**

Android Studio + un AVD déjà créé (sinon ça explose le timing).

#### Concept clé

Une app React Native n'est pas une SPA navigateur : pas de CORS navigateur, mais un vrai runtime mobile.

“localhost” n'est pas la même machine : sur un téléphone/émulateur, localhost ≠ votre PC. (Je vous donne un mapping clair dans le Bloc B.)

## Bloc B (55 min) — Créer le projet mobile + Secure Storage

On veut une mini architecture lisible (peu de fichiers, stable).

### B1 Arborescence cible (comme vos TPs : “où mettre quoi”)

**NE FAIS RIEN, NE CREE RIEN !!!** C'est un aperçu de l'arborescence qu'on va avoir à la fin :

```
mobile/  
  App.js  
  src/  
    config.js  
    api/apiClient.js  
    auth/AuthContext.js  
    screens/LoginScreen.js  
    screens/PagesListScreen.js  
    screens/PageDetailScreen.js
```

### B2 Créer l'app Expo

Depuis la racine mycellius/:

```
npx create-expo-app mobile      # il va faire tout seul un npm install  
cd mobile
```

### B3 Installer les libs minimales (navigation + secure store)

```
npx expo install expo-secure-store  
npm install @react-navigation/native @react-navigation/native-stack  
npx expo install react-native-screens react-native-safe-area-context
```

Pourquoi ces libs (concepts)

- Navigation : sur mobile, on raisonne en “écrans” (Login, Liste, Détail), pas en routes navigateur.
- SecureStore : stockage chiffré/OS-level, mieux que AsyncStorage pour un token.

### B4 Configurer l'URL d'API (le point le plus chronophage)

Créer : mobile/src/config.js

```
// ⚠ IMPORTANT : l'URL dépend de l'environnement d'exécution  
// Cas 1 : Android Emulator => 10.0.2.2 pointe vers le "localhost" du PC  
// Cas 2 : iOS Simulator (Mac) => localhost fonctionne souvent  
// Cas 3 : Smartphone réel => utiliser l'IP LAN du PC (ex: http://192.168.1.20:8080)
```

```
export const API_URL = "http://TON\_IP\_PC:8080";    #moi c'est 192.168.1.21
```

Règle simple

“Mon app tourne où ?” → “Donc elle voit quel réseau ?”

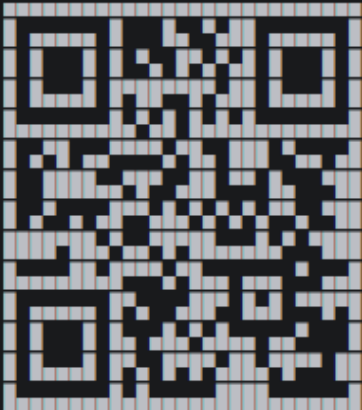
Si tu bosses sur ton smartphone réel : remplacez par l’IP du PC (même Wi-Fi).

**Sur ton iPhone/Android, installe juste l'app “Expo Go”, puis scanne le QR code après avoir fait : npm run start**

```
PS C:\Users\clakg\mycellius\mobile> npm run start

> mobile@1.0.0 start
> expo start

Starting project at C:\Users\clakg\mycellius\mobile
React Compiler enabled
Starting Metro Bundler



> Metro waiting on exp://192.168.1.21:8081
> Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

> Web is waiting on http://localhost:8081

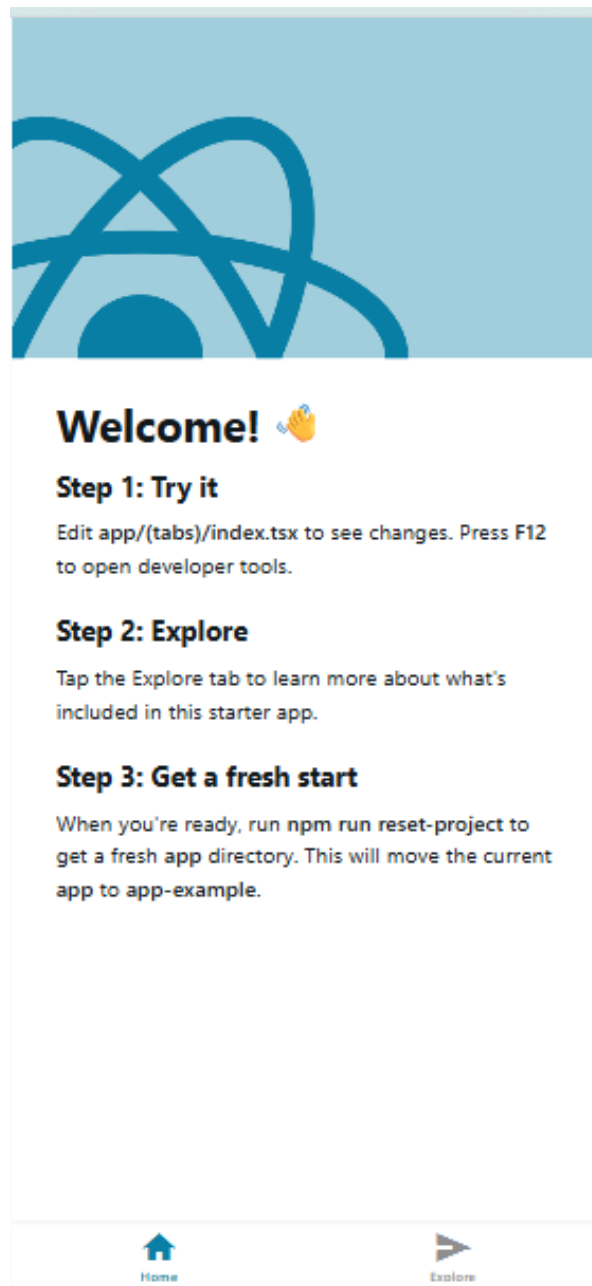
> Using Expo Go
> Press s | switch to development build

> Press a | open Android
> Press w | open web

> Press j | open debugger
> Press r | reload app
> Press m | toggle menu
> shift+m | more tools
> Press o | open project code in your editor

> Press ? | show all commands
```

=> Tu obtiens un affichage de ton application sur ton téléphone via Expo



Et voilààààà !!!!!

Comme pour beaucoup de framework, des templates sont déjà créés. Vous pouvez jeter un coup d'oeil au code évidemment.

C'est beau tout ça mais va bien falloir utiliser l'API qu'on s'est donné du mal à faire !

On va créer :

- api/apiClient.js,
- auth/AuthContext.js,
- puis les 3 écrans (Login, Liste, Détail).

## B5 Client HTTP unique (comme le web)

**Objectif :** centraliser fetch + Bearer + erreurs (même logique que votre apiClient côté web).

Créer : mobile/src/api/apiClient.js

```
import { API_URL } from "../config";

export class ApiError extends Error {
  constructor(status, body) {
    super(`API error ${status}`);
    this.status = status;
    this.body = body;
  }
}

export async function apiRequest(path, { method = "GET", token = null, body = null } = {}) {
  const headers = { "Content-Type": "application/json" };
  if (token) headers.Authorization = `Bearer ${token}`;

  const res = await fetch(`${API_URL}${path}`, {
    method,
    headers,
    body: body ? JSON.stringify(body) : null,
  });

  const text = await res.text();
  let data = null;
  try { data = text ? JSON.parse(text) : null; } catch { data = text; }

  if (!res.ok) throw new ApiError(res.status, data);
  return data;
}
```

## B6 AuthContext “mobile” (US-H1 : token en SecureStore)

**Objectif :**

au démarrage, on relit le token (onboarding). Au login, on le stocke. Au logout, on l’efface.

(Concept = même idée que votre AuthContext web, mais stockage différent).

Créer : mobile/src/auth/AuthContext.js

```
import React, { createContext, useContext, useEffect, useMemo, useState } from "react";
import * as SecureStore from "expo-secure-store";

const AuthContext = createContext(null);

const TOKEN_KEY = "mycellius_token";
const ROLE_KEY = "mycellius_role";
const USERNAME_KEY = "mycellius_username";

export function AuthProvider({ children }) {
  const [booting, setBooting] = useState(true);
  const [token, setToken] = useState(null);
  const [role, setRole] = useState(null);
  const [username, setUsername] = useState(null);
```

```

// Onboarding : relire le token au démarrage
useEffect(() => {
  (async () => {
    const t = await SecureStore.getItemAsync(TOKEN_KEY);
    const r = await SecureStore.getItemAsync(ROLE_KEY);
    const u = await SecureStore.getItemAsync(USERNAME_KEY);
    setToken(t);
    setRole(r);
    setUsername(u);
    setBooting(false);
  })();
}, []);

const value = useMemo(() => ({
  booting,
  token,
  role,
  username,
  login: async ({ token, role, username }) => {
    await SecureStore.setItemAsync(TOKEN_KEY, token);
    await SecureStore.setItemAsync(ROLE_KEY, role);
    await SecureStore.setItemAsync(USERNAME_KEY, username);
    setToken(token);
    setRole(role);
    setUsername(username);
  },
  logout: async () => {
    await SecureStore.deleteItemAsync(TOKEN_KEY);
    await SecureStore.deleteItemAsync(ROLE_KEY);
    await SecureStore.deleteItemAsync(USERNAME_KEY);
    setToken(null);
    setRole(null);
    setUsername(null);
  }
}), [booting, token, role, username]);

return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  return useContext(AuthContext);
}

```

## Check Bloc B

- > L'app démarre sans crash.
- > On a un client HTTP + un AuthProvider prêt.

## Bloc C (55 min) — Ecrans : login + liste + détail + recherche

### C1 Navigation + “garde” (RequireAuth version mobile)

Fichier : mobile/App.js

```
import React from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import { AuthProvider, useAuth } from "../src/auth/AuthContext";

import LoginScreen from "../src/screens/LoginScreen";
import PagesListScreen from "../src/screens/PagesListScreen";
import PageDetailScreen from "../src/screens/PageDetailScreen";

const Stack = createNativeStackNavigator();

function AppNavigator() {
  const { booting, token } = useAuth();

  if (booting) return null; // option : Splash/loader

  return (
    <Stack.Navigator>
      {!token ? (
        <Stack.Screen name="Login" component={LoginScreen} options={{ title: "Mycellius
— Login" }} />
      ) : (
        <>
          <Stack.Screen name="Pages" component={PagesListScreen} options={{ title:
"Pages" }} />
          <Stack.Screen name="PageDetail" component={PageDetailScreen} options={{ title:
"Détail" }} />
        </>
      )}
    </Stack.Navigator>
  );
}

export default function App() {
  return (
    <AuthProvider>
      <NavigationContainer>
        <AppNavigator />
      </NavigationContainer>
    </AuthProvider>
  );
}
```

Concept : onboarding

- > Si token existe dans SecureStore → on va direct sur “Pages”
- > Sinon → écran Login



## C2 Ecran Login (appel POST /api/v1/auth/login)

**Objectif identique au web** : récupérer {token, username, role}.

Créer : mobile/src/screens/LoginScreen.js

```
import React, { useState } from "react";
import { View, Text, TextInput, Button } from "react-native";
import { apiRequest, ApiError } from "../api/apiClient";
import { useAuth } from "../auth/AuthContext";

export default function LoginScreen() {
  const [username, setUsername] = useState("admin");
  const [password, setPassword] = useState("Admin123!");
  const [error, setError] = useState(null);

  const auth = useAuth();

  async function onLogin() {
    setError(null);
    try {
      const res = await apiRequest("/api/v1/auth/login", {
        method: "POST",
        body: { username, password },
      });
      await auth.login(res);
      // La navigation bascule automatiquement (token présent)
    } catch (e) {
      if (e instanceof ApiError) setError(typeof e.body === "string" ? e.body : "Login failed");
      else setError("Login failed");
    }
  }

  return (
    <View style={{ padding: 16, gap: 12 }}>
      <Text>Username</Text>
      <TextInput value={username} onChangeText={setUsername} autoCapitalize="none"
        style={{ borderWidth: 1, padding: 8 }} />

      <Text>Password</Text>
      <TextInput value={password} onChangeText={setPassword} secureTextEntry
        style={{ borderWidth: 1, padding: 8 }} />

      <Button title="Se connecter" onPress={onLogin} />
      {error && <Text style={{ marginTop: 8 }}>{String(error)}</Text>}
    </View>
  );
}
```

## C3 Ecran Liste + recherche (US-H2)

### Contraintes d'API (comme dans votre front web)

> Variante A : GET /api/v1/pages?page=0&size=10 → réponse { content: [...] }

> Variante B : GET /api/v1/pages → réponse [...]

> Recherche : idéalement /api/v1/pages/search?title=xxx (hérité de vos séances API)

Créer : mobile/src/screens/PagesListScreen.js

```
import React, { useEffect, useState } from "react";
import { View, Text, TextInput, Button, FlatList, TouchableOpacity } from "react-native";
import { apiRequest, ApiError } from "../api/apiClient";
import { useAuth } from "../auth/AuthContext";

export default function PagesListScreen({ navigation }) {
  const { token, role, logout } = useAuth();
  const [pages, setPages] = useState([]);
  const [query, setQuery] = useState("");
  const [error, setError] = useState(null);

  async function loadList() {
    setError(null);
    try {
      let res;
      try {
        res = await apiRequest("/api/v1/pages?page=0&size=50", { token });
      } catch {
        res = await apiRequest("/api/v1/pages", { token });
      }
      const content = Array.isArray(res) ? res : (res?.content ?? []);
      setPages(content);
    } catch (e) {
      if (e instanceof ApiError && e.status === 401) { await logout(); return; }
      setError("Erreur chargement pages");
    }
  }

  async function onSearch() {
    setError(null);
    const q = query.trim();
    if (!q) { loadList(); return; }

    try {
      // Recherche "titre contient"
      // Si votre API expose /api/v1/pages/search?title=..., c'est parfait
      const res = await apiRequest(`/api/v1/pages/search?title=${encodeURIComponent(q)}`, { token });
      const content = Array.isArray(res) ? res : (res?.content ?? []);
      setPages(content);
    } catch (e) {
      // Si l'endpoint search n'existe pas encore, on retombe sur la liste locale filtrée
      // (Option de secours pédagogique : pas idéale métier, mais évite de bloquer la
séance)
      try {
        await loadList();
      }
    }
  }
}
```

```

        setPages(prev => prev.filter(p => (p.title ??
        "").toLowerCase().includes(q.toLowerCase())));
    } catch {
        setError("Recherche impossible (endpoint absent + liste KO)");
    }
}
}

useEffect(() => { loadList(); }, [token]);

return (
    <View style={{ padding: 16, gap: 12 }}>
        <View style={{ flexDirection: "row", gap: 8 }}>
            <TextInput placeholder="Rechercher par titre..." value={query}
onChangeText={setQuery}
            style={{ borderWidth: 1, padding: 8, flex: 1 }} />
            <Button title="OK" onPress={onSearch} />
        </View>

        <Button title="Rafraîchir" onPress={loadList} />
        {error && <Text>{error}</Text>}

        <FlatList
            data={pages}
            keyExtractor={({item}) => String(item.id)}
            renderItem={({ item }) => (
                <TouchableOpacity onPress={() => navigation.navigate("PageDetail", { id: item.id })}>
                    <Text style={{ paddingVertical: 10 }}>
                        {item.title} ({item.id})
                    </Text>
                </TouchableOpacity>
            )}
        />
    </View>
);
}

```

## C4 Ecran Détail (lecture /api/v1/pages/{id})

Créer : mobile/src/screens/PageDetailScreen.js

```
import React, { useEffect, useState } from "react";
import { View, Text, Button, ScrollView } from "react-native";
import { apiRequest, ApiError } from "../api/apiClient";
import { useAuth } from "../auth/AuthContext";

export default function PageDetailScreen({ route, navigation }) {
  const { id } = route.params;
  const { token, logout } = useAuth();
  const [page, setPage] = useState(null);

  useEffect(() => {
    (async () => {
      try {
        const res = await apiRequest(`/api/v1/pages/${id}`, { token });
        setPage(res);
      } catch (e) {
        if (e instanceof ApiError && e.status === 401) { await logout(); return; }
      }
    })();
  }, [id, token]);

  return (
    <ScrollView style={{ padding: 16 }}>
      {!page ? (
        <Text>Chargement...</Text>
      ) : (
        <>
          <Text style={{ fontSize: 20, fontWeight: "700" }}>{page.title}</Text>
          <Text style={{ marginTop: 8 }}>ID : {page.id}</Text>
          <Text style={{ marginTop: 16, lineHeight: 20 }}>{page.content}</Text>
        </>
      )}
      <View style={{ marginTop: 20 }}>
        <Button title="Retour" onPress={() => navigation.goBack()} />
      </View>
    </ScrollView>
  );
}
```

### Checkpoint Bloc C

- > Login OK → bascule automatique vers Liste (token stocké).
- > Liste charge des pages (Variante A ou B).
- > Détail affiche une page.
- > Recherche : si endpoint search existe, elle tape /search ; sinon fallback filtre local (pour ne pas bloquer la séance).

## Bloc D (15 min) — Preuves, captures, checklist test

Ce bloc est “non négociable” : c’est ce qui prouve la séance (comme vos preuves Postman / Web).

**Captures écran minimales à fournir pour votre dossier E6 par exemple** (dans un dossier docs/seance9/)

- > Écran Login (avant connexion)
- > Écran Pages (liste) après connexion
- > Écran Détail d’une page
- > Recherche : avant/après (liste filtrée)

### Checklist de test fonctionnel

- API démarrée + DB OK (docker ps)
- POST /api/v1/auth/login renvoie un token (Postman)
- Mobile : login admin OK → l’app arrive sur la liste
- Fermer/réouvrir l’app → l’app reste connectée (preuve SecureStore)
- Liste : au moins 1 page visible
- Détail : lecture d’une page OK
- Recherche : “onboarding” renvoie une liste filtrée (via endpoint search ou fallback)
- Logout (si vous l’ajoutez en bonus) : retour écran Login + token supprimé

### Bonus “si ça bloque” (3 erreurs typiques, correction immédiate)

#### 1. “Network request failed”

90% du temps : API\_URL faux.

Android Emulator : <http://10.0.2.2:8080>

Smartphone : [http://IP\\_DU\\_PC:8080](http://IP_DU_PC:8080)

#### 2. Login OK sur Postman mais KO sur mobile

même cause : URL/réseau.

ou téléphone pas sur le même Wi-Fi que le PC.

#### 3. 401 sur /api/v1/pages

token absent / pas envoyé.

vérifier que apiClient met bien Authorization: Bearer <token> (c’est le standard de votre sécu).

Alors vous avez beau recharger la page sur votre tel mais ca ne change pas l'affichage...  
Je vous laisse chercher un peu...

Dans le fichier package.json, l'entrée de notre app n'est pas correcte.

En ligne 3, tu as :

```
"main": "expo-router/entry"
```

Remplace le par :

```
"main": "expo/AppEntry.js"
```

### **Pourquoi ?**

Ton projet est en mode Expo Router (d'où le app/index.tsx). Si tu veux suivre le TP "App.js + React Navigation", il faut juste faire démarrer Expo sur App.js.